

1.Vibe coding to build Front end JIRA integration to fetch US stories

CONNECT JIRA BUTTON IMPLEMENTATION

Start Over

Read my entire workspace and understand how UI implemented in the frontend folder

I want to add "Connect Jira" button to establish the connection(Base url,Email id,Jira API key_)

once the jira is connected userstories should be populated in the UI screen.

[STRICTLY] DO THE IMPLEMENTATION ONLY ON UI

Inspecting frontend files

Reviewing the frontend implementation and current API integration before making UI-only changes.

Reviewed 4 files and evaluated implementation options

Implementing the Jira UI controls and story population flow in the frontend only.

Updated api.ts +1 -1

I've partially updated the API helpers; checking file contents now to finish the exact UI-only Jira implementation cleanly.

Reviewed 2 files and fixed import issues

Patching the frontend type definition for Jira stories.

Considering file issues

CHAT

CONNECT JIRA BUTTON IMPLEMENTATION

Ran cd /Users/nisha/Documents/user-story-to-tests/frontend

Verified code changes and discussed new UI updates

UI Update Complete

Implemented the Jira connection flow entirely in the frontend.

Changes made

- App.tsx
 - Added a new Jira connection panel with inputs for:
 - Jira Base URL
 - Email
 - Jira API Key
 - Added a Connect Jira button and connection status messaging
 - Added Jira story cards to display fetched Story issues
 - Added Use this story button for each Jira issue to populate the form title/description
- api.ts
 - Added fetchJiraStories() helper to call Jira REST API directly from the frontend
- types.ts
 - Added JiraStory type

Validation

- Ran npm run typecheck in frontend successfully with no TypeScript errors.

This implementation is UI-only and uses the browser to fetch Jira stories using the provided Jira credentials.

> 2 files changed +220 -6

Keep Undo

tsconfig.json

EXPLORER

USER-STOR...

backend

src

package.json

tsconfig.json

frontend

node_modules

src

api.ts

App.tsx

main.tsx

types.ts

vite-env.d.ts

.env

index.html

package.json

tsconfig.json

tsconfig.node.json

vite.config.ts

node_modules

.env

.gitignore

package-lock.json

package.json

.env

TS types.ts

frontend > src > TS types.ts > => GenerateRequest

6 }
7
8 export interface TestCase {
9 id: string
10 title: string
11 steps: string[]
12 testData?: string
13 expectedResult: string
14 category: string
15 }
16
17 export interface GenerateResponse {
18 cases: TestCase[]
19 model?: string
20 promptTokens: number
21 completionTokens: number
22 }
23
24 export interface JiraStory {
25 id: string
26 key: string
27 summary: string
28 description?: string
29 status: string
30 issueType?: string
31 }

PROBLEMS 2

OUTPUT

DEBUG CONSOLE

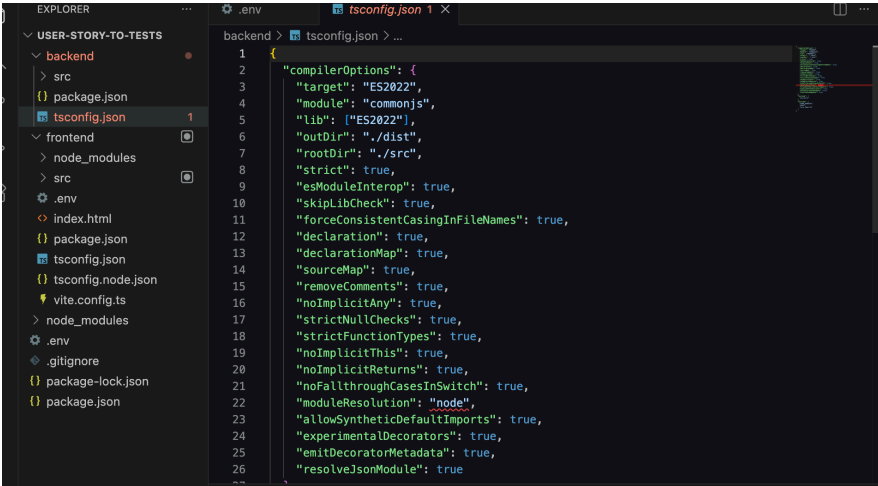
TERMINAL

PORTS

zsh

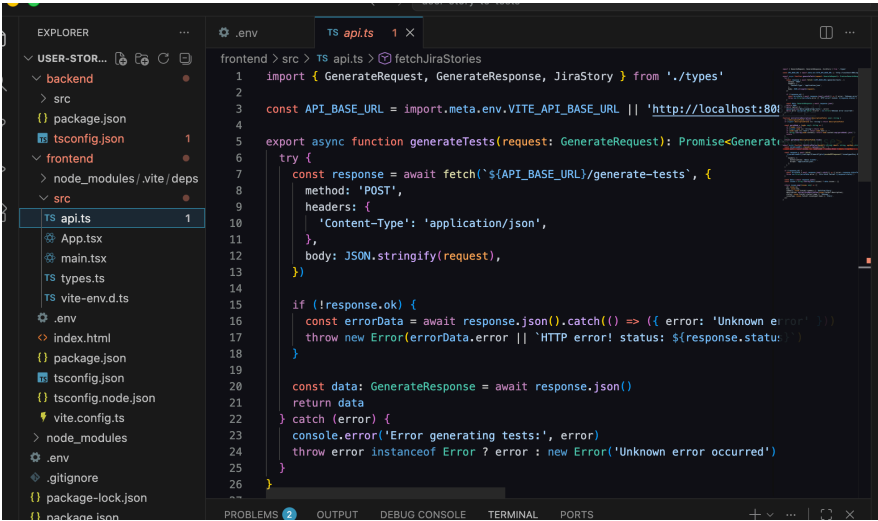
node back...

Post code changes



The screenshot shows the VS Code interface with the Explorer on the left and the Editor on the right. The Explorer shows a project structure with a 'backend' directory containing 'src', 'package.json', and 'tsconfig.json'. The 'tsconfig.json' file is selected and its content is displayed in the Editor. The content is a JSON object with various compiler options.

```
1 {
2   "compilerOptions": {
3     "target": "ES2022",
4     "module": "commonjs",
5     "lib": ["ES2022"],
6     "outDir": "./dist",
7     "rootDir": "./src",
8     "strict": true,
9     "esModuleInterop": true,
10    "skipLibCheck": true,
11    "forceConsistentCasingInFileNames": true,
12    "declaration": true,
13    "declarationMap": true,
14    "sourceMap": true,
15    "removeComments": true,
16    "noImplicitAny": true,
17    "strictNullChecks": true,
18    "strictFunctionTypes": true,
19    "noImplicitThis": true,
20    "noImplicitReturns": true,
21    "noFallthroughCasesInSwitch": true,
22    "moduleResolution": "node",
23    "allowSyntheticDefaultImports": true,
24    "experimentalDecorators": true,
25    "emitDecoratorMetadata": true,
26    "resolveJsonModule": true
```



The screenshot shows the VS Code interface with the Explorer on the left and the Editor on the right. The Explorer shows a project structure with a 'frontend' directory containing 'src', 'package.json', 'tsconfig.json', 'vite.config.ts', and 'vite-env.d.ts'. The 'ts api.ts' file is selected and its content is displayed in the Editor. The content is a TypeScript file with imports and a function definition.

```
1 import { GenerateRequest, GenerateResponse, JiraStory } from './types'
2
3 const API_BASE_URL = import.meta.env.VITE_API_BASE_URL || 'http://localhost:8080'
4
5 export async function generateTests(request: GenerateRequest): Promise<GenerateResponse> {
6   try {
7     const response = await fetch(`${API_BASE_URL}/generate-tests`, {
8       method: 'POST',
9       headers: {
10        'Content-Type': 'application/json',
11      },
12      body: JSON.stringify(request),
13    })
14
15    if (!response.ok) {
16      const errorData = await response.json().catch(() => ({ error: 'Unknown error' }))
17      throw new Error(errorData.error || 'HTTP error! status: ${response.status}')
18    }
19
20    const data: GenerateResponse = await response.json()
21    return data
22  } catch (error) {
23    console.error('Error generating tests:', error)
24    throw error instanceof Error ? error : new Error('Unknown error occurred')
25  }
26 }
```

Verify in the UI

localhost:5173

☆

Connect Jira

Enter your Jira base URL, email, and API key to fetch Story issues directly into the app.

Jira Base URL

Email

Jira API Key

https://your-domain.atlassian.net

user@example.com

API token

Connect Jira

Story Title *

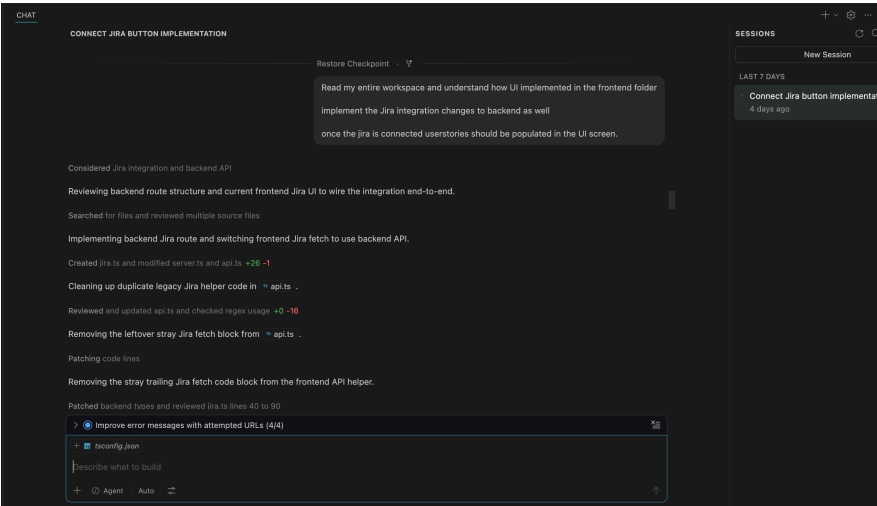
Enter the user story title...

Description

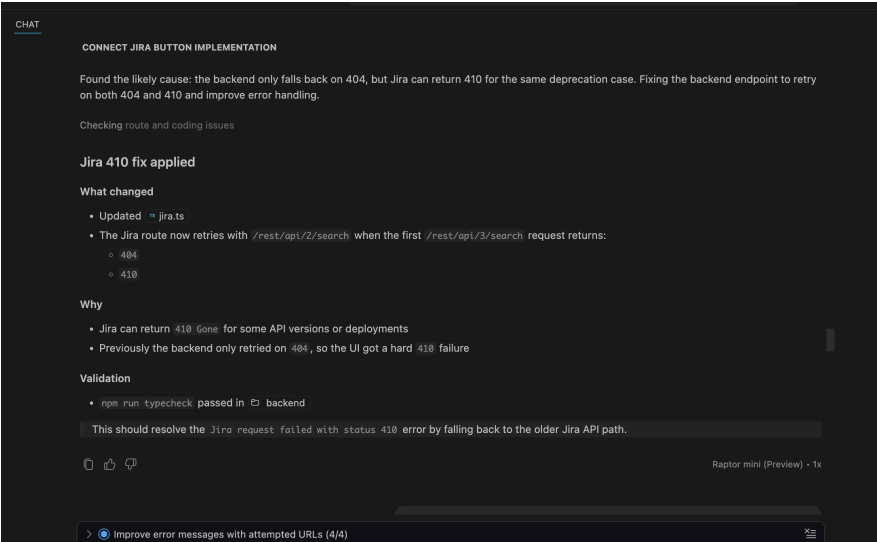
Additional description (optional)...

Acceptance Criteria *

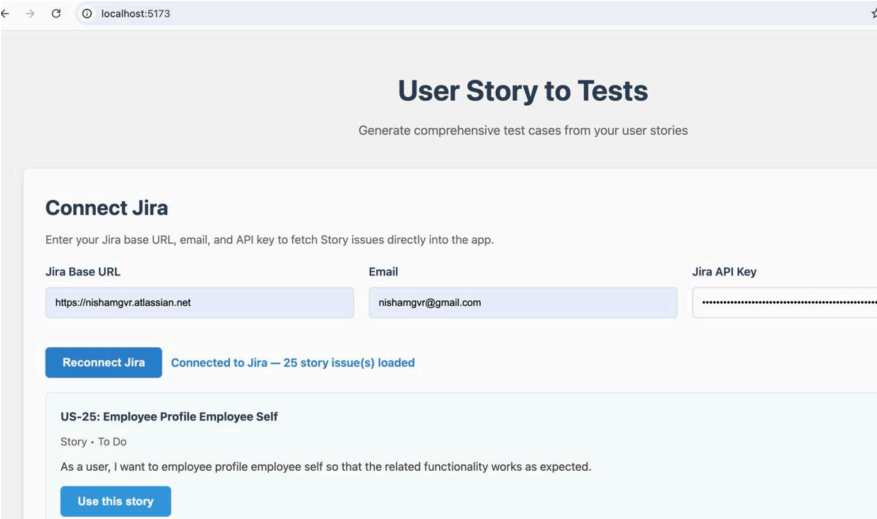
Integrate Backend for Jira Connectivity



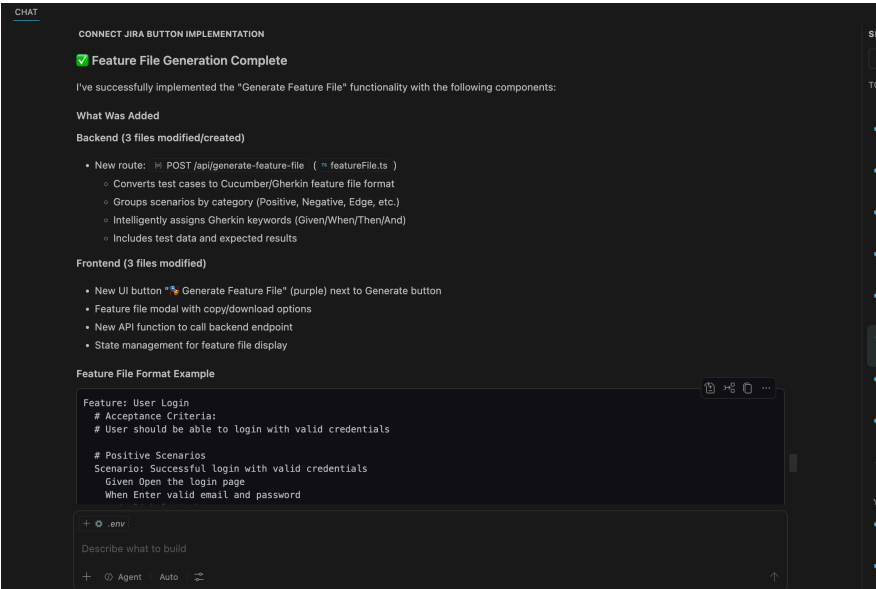
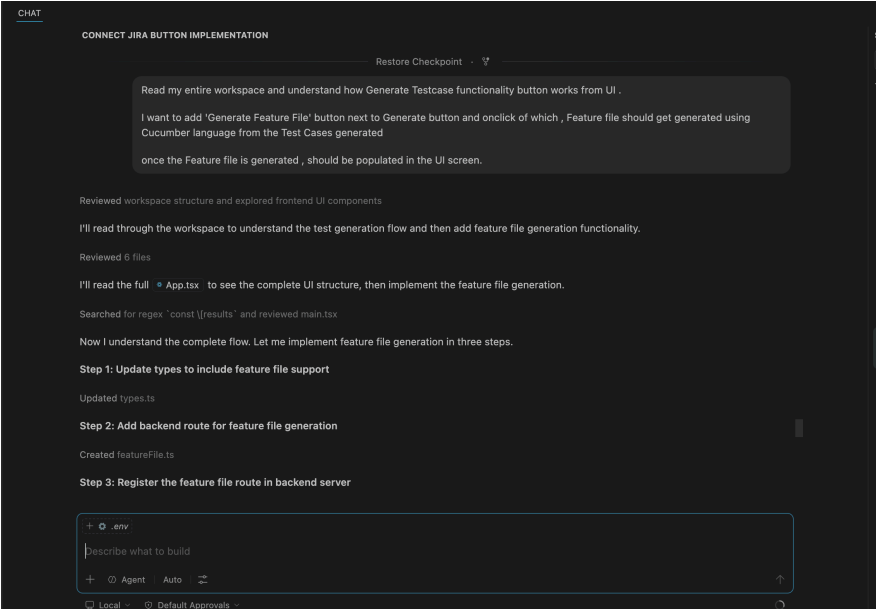
Implemented chain of thoughts to fix error in the code



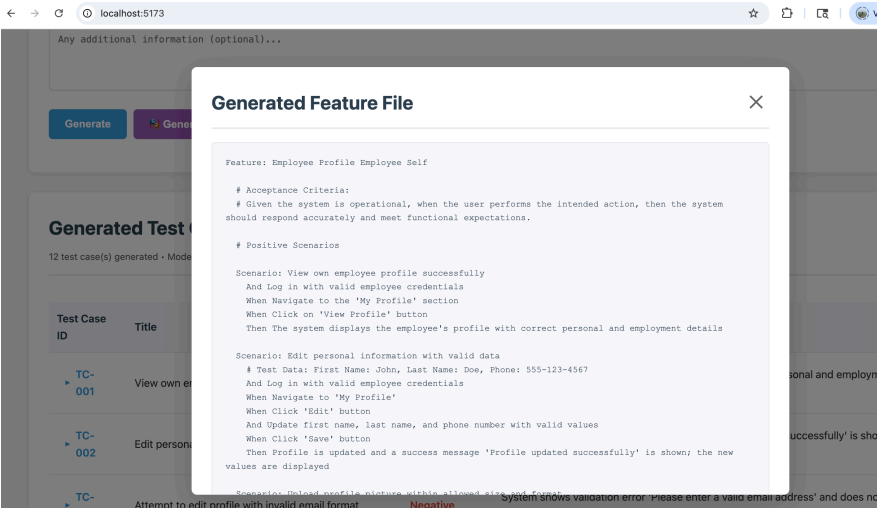
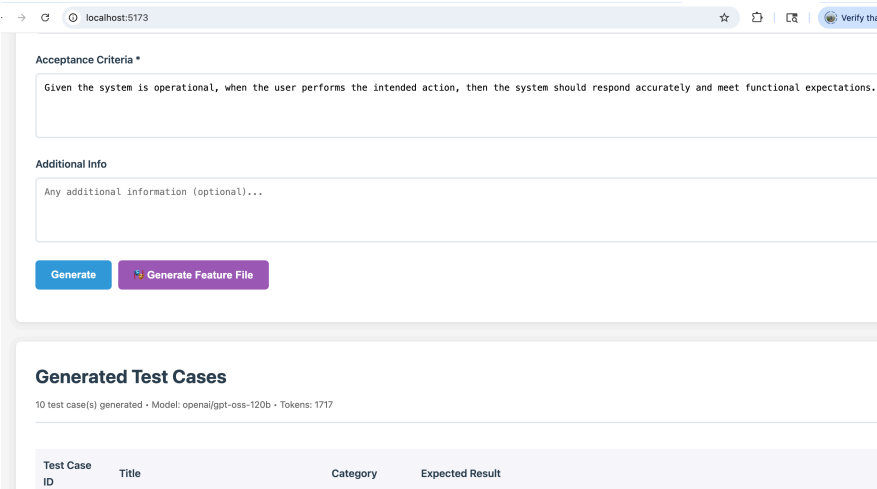
Successfully Displayed user Stories in the UI



2. Prompt to add Feature file button functionality



Implemented the Feature file button functionality



← → localhost:5173

Any additional information (optional)...

Generate

Generated Test

12 test case(s) generated - Mode

Test Case ID	Title
TC-001	View own e
TC-002	Edit person
TC-003	Attempt to edit profile with invalid email format

When In Browser 1, change phone number and click 'Save'

When In Browser 2, change address and click 'Save'

Then Browser 2 receives a warning 'Profile has been updated by another session. Please reload and try again.'

Scenario: Handle special characters in text fields

And log in with valid employee credentials

When Navigate to 'My Profile' and click 'Edit'

When Enter 'Q'Conner' in Last Name field and 'São Paulo' in City field

When Click 'Save' button

Then Profile updates successfully and displays the special characters correctly without data corruption

Non-Functional Scenarios

Scenario: Profile page load performance under normal load

And log in with valid employee credentials

When Navigate to 'My Profile' section

Then Profile page loads and renders within 2 seconds under standard network conditions

Scenario: Profile page accessibility compliance

And log in with valid employee credentials

When Navigate to 'My Profile'

And Run accessibility audit tool (e.g., axe) on the page

Then No WCAG 2.1 Level AA violations are reported

Copy to Clipboard

Download

Close

personal and employment

successfully' is shown; the

system shows validation error 'Please enter a valid email address' and does not save changes